Leveraging Model Fusion for Improved License Plate Recognition

The paper, our re-run on the fixed dataset (v1.1), whether the run is OK, how it fits our research, and what to show and say to the faculty

Prepared for MD. Rokib Hasan Oli (paper reader) · Supervisor: Dr. Md Mansur ul Mulk · 24 October 2026
Paper: R. Laroca, L. A. Zanlorensi, V. Estevam, R. Minetto and D. Menotti (Federal University of Paraná, Brazil), *Iberoamerican Congress on Pattern Recognition (CIARP 2023)*, Springer LNCS, pp. 60-75, DOI 10.1007/978-3-031-49249-5_5. Author version arXiv 2309.04331v2, 15 pages; page numbers below are its pages.
Our run: Kaggle notebook `paper24_kaggle.ipynb` on Tesla T4 x2, 22 Oct 2026, dataset **v1.1**, 79 min, all 5 jobs `code=0`. Outputs: `After Train/results.zip`. Every number of ours was recomputed from the per-plate predictions in that zip (`report_src/analysis.py`) and the two shared models were re-scored on this PC (`report_src/local_check.py`). This report replaces the one of 21 Oct (v1.0 run), kept as `Paper24_Study_Report_v1.0_run.pdf`.

The whole report on one page

The paper in three lines. Read the same plate crop with several different OCR models and combine their answers. The best rule is a plain majority vote over whole strings (ties broken by confidence or by the best model). With 12 models on 8 datasets the mean plate accuracy rises from 92.4 % (best single model) to 97.6 % (p.9), and on 4 unseen datasets from 87.6 % to 90.3 % (p.11). Trusting the most confident model is the worst rule. Fusing 4-6 fast models gives most of the gain for a fraction of the time (p.12).

Is the v1.1 run OK? Yes. It used dataset v1.1 and the current kit code; every job finished all its epochs; all 66 fusion rows on Kaggle match our recomputation; and the two models we share on GitHub give the same readings on this PC as on Kaggle (all 651 test plates for char-YOLO, 648 of 651 for CRNN, the same accuracy). The dataset fix worked: the 19 BLPD test rows that were whole car photos are now plate crops, and char-YOLO reads 79 % of them instead of 5 %.

What the run found. (1) **The paper's recipe still gives nothing on our data:** the best voting rule chosen on validation reads 86.33 % against 86.79 % for char-YOLOv8n alone (-0.46 points: 5 plates fixed, 8 broken, p = 0.58). (2) **Why:** the models fail together. On 65 of the 86 plates char-YOLO gets wrong, every other model is wrong too, so no vote can help (ceiling: 90.02 %). (3) **Confidence cannot be compared across models:** CRNN gives 65 % of its wrong readings a confidence of 0.9 or more, so "take the most confident" drops to about 76 %. (4) **Our Bangla-specific rule works, again:** keep char-YOLO's reading when it fits the BRTA plate layout, otherwise ask CRNN. Every reading that broke the layout was wrong (46 of 46 on test, 92 of 92 on validation). Result: **88.63 % (+1.84 points, 12 fixed, 0 broken, p = 0.0005)** at 9.8 ms per plate. The gain and the 12 / 0 split are the same as in the v1.0 run, so the result **replicated**. (5) **New:** on real crops, char-YOLO's errors are mostly a digit boxed twice or a letter read as two classes (39 of 86 wrong plates), so the next gain is in its decoder.

What changed since the first report. All scores moved because 19 test images are now real crops (char-YOLO 85.56 → 86.79 %). The conclusions held. Two details did not: the fallback no longer helps at night (it now helps most on blurry plates), and on the 632 unchanged plates the retrained models moved by 1-3 points between the two runs (the slot CNN even by +10.6). So one run cannot rank two models that are 1-2 points apart; we need seeds (section 11).

One sentence for the faculty. "We re-ran the paper's model-fusion idea on our corrected Bangla dataset: plain voting still does not help because our models make the same mistakes, but using the BRTA plate format to decide when to ask a second model gives a significant +1.8 points at almost no cost, and it replicated across two runs."

Layout. Part A (sections 1-7): understand the paper. Part B (8-14): our experiment; read it before you quote our numbers. Part C (15-21): research use, benefits, slides, script, questions, next steps, files.

1. What problem does the paper solve?

A plate reader has two stages: find the plate (detection) and read its characters (recognition, OCR). Detection is nearly solved by YOLO-type models, so recent work focuses on recognition (p.1-2). But earlier studies, several by the same group, showed that recognition models are **uneven**: a model that is best on one dataset can be poor on another, because each dataset has its own layouts, fonts, tilt and image quality (p.2).

So the paper asks three plain questions (p.2):

1. Can we get clearly better plate reading by **combining the outputs** of several different recognition models?
2. If yes, **how much** better?
3. **How many and which** models should we combine?

Before this paper, LPR fusion work had used copies of one model (Izidio et al., several Tiny-YOLOv3 nets on a private set) or synthetic degraded images (Schirrmacher et al.); nobody had combined many *different* models across many public datasets (p.4). The authors also stress a practical point: some applications (traffic tickets, forensics) prefer accuracy over speed, so they can afford several models (p.2).

2. Key terms in plain words

Term	Meaning
Model fusion / ensemble	Run several models on the same input and combine their answers. It helps only when the models make *different* mistakes.
Recognition rate (plate exact)	Share of plates whose **whole** string is right. One wrong character = wrong plate (p.8). This is the paper's only metric.
HC (highest confidence)	Pick the string of the model that is most sure of itself, even if no other model agrees (p.7).
MV (majority vote)	Pick the full string that most models output. Confidence is ignored unless there is a tie (p.7).
MVCP (vote per character position)	Vote separately on character 1, character 2, ... and join the winners. Breaks when one model drops or adds a character, because the positions shift (p.10-11).
-HC / -BM tie-break	If two strings get the same number of votes: take the more confident one (-HC) or the one from the "best model" (-BM) (p.7-8).
Top-N	Fuse the N best models (ranked by mean accuracy). Table 4 tries N = 2 ... 12.
Intra-dataset / cross-dataset	Test on the test split of a dataset used in training / on a dataset never seen in training (harder, closer to a new camera).
Oracle (our term)	A plate counts as right if *any* model is right. No voting rule can beat it, so it is the ceiling for fusion.
Calibration (our term)	A model is calibrated if, among readings with confidence 0.9, about 90 % are right. Uncalibrated confidences make HC fail.
BRTA layout (our term)	The format of a Bangladeshi plate: one district word, optional মেট্রো, one series letter, six digits. A reading that breaks it cannot be right.
McNemar test (our term)	The standard test for "reader A beats reader B on the same plates": it only looks at the plates where they disagree (fixed vs broken).

3. The 12 models and the 12 datasets

The models were chosen because they cover the design patterns of modern text recognisers and are common baselines in LPR (p.5). Each was trained **once** on the union of 8 training sets, in its own original framework:

Family	Models	How they read	Framework, training (p.5)
Scene-text, CTC	CRNN, GRCNN, Rosetta, STAR-Net	CNN features, (BiLSTM), then CTC decoding	PyTorch (Clova AI fork), Adadelta, 300 k iterations, batch 128
Scene-text, attention	R2AM, RARE, TRBA	CNN, (TPS rectification), BiLSTM, attention decoder	same
Transformer	ViTSTR-Base	ViT patches; output tokens are the characters	same
Multi-task CNN	Holistic-CNN, Multi-Task-LR	CNN, then one classifier per character slot	Keras, Adam 1e-3, batch 64, early stop

YOLO character detectors	CR-NET, Fast-OCR	detect every character as an object, sort the boxes	Darknet, SGD, 90 k iterations, batch 64

Data (p.5-7, Table 2). 8 datasets for training and testing: Caltech Cars, EnglishLP, UCSD-Stills, ChineseLP, AOLP, SSIG-SegPlate, UFPR-ALPR, RodoSol-ALPR (20,000 images; Brazilian and Mercosur plates). 4 datasets only for cross-dataset tests: OpenALPR-EU, PKU, CD-HARD (heavily tilted, mixed layouts) and CLPD. Training also used 772 web images and synthetic plates made by **shuffling character positions**, so that models cannot memorise sequences (p.6-7). Crops come from a YOLOv4 detector, not from the ground truth, to be realistic (p.8).

> **Mapping to our run.** We trained one model from each of four of these families on our data: char-YOLOv8n (the CR-NET / Fast-OCR family), CRNN-CTC (CRNN), a slot CNN in the fast-plate-ocr style (Holistic-CNN / Multi-Task-LR) and ViTSTR-tiny (ViTSTR). Section 8 gives the details.

4. The method: three rules and two tie-breaks

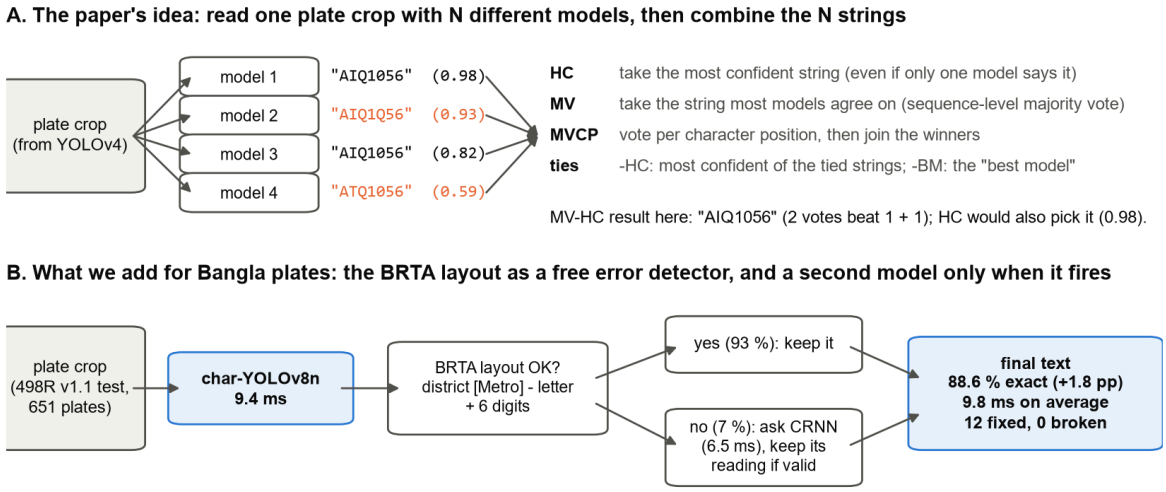


Figure 1. A: the paper's fusion (example strings from its Fig. 4, p.10; orange = wrong). B: the change we test on Bangla plates (section 12, finding 4). Numbers in B are ours, on the 498R v1.1 test split.

The whole method fits in a few lines. For each plate, every model gives a string and a confidence. Then:

- **HC:** output the string with the highest confidence (p.7).
- **MV:** group identical strings and output the largest group. In the paper's example, five models say "ABC-123", "ABC-123", "ABC-124", "ABC-124", "ABC-125": a tie between two strings (p.7).
- **MVCP:** vote on every character position and join the winners (p.7).
- **Ties:** -HC takes the tied string with the higher confidence; -BM takes the tied string that the best single model output (p.7-8). With only **two** models every disagreement is a tie, so MV-BM is just "trust the best model" and MV-HC is just HC. You need at least three models before voting does anything.
- **Confidences are used raw.** The authors tried normalising them per model; it did not help (p.8).

The "best model" for -BM and the top-N order are taken from the mean accuracy over the datasets. The authors say the validation ranking is "essentially the same, with only two models swapping positions" (p.9).

5. Every result in the paper

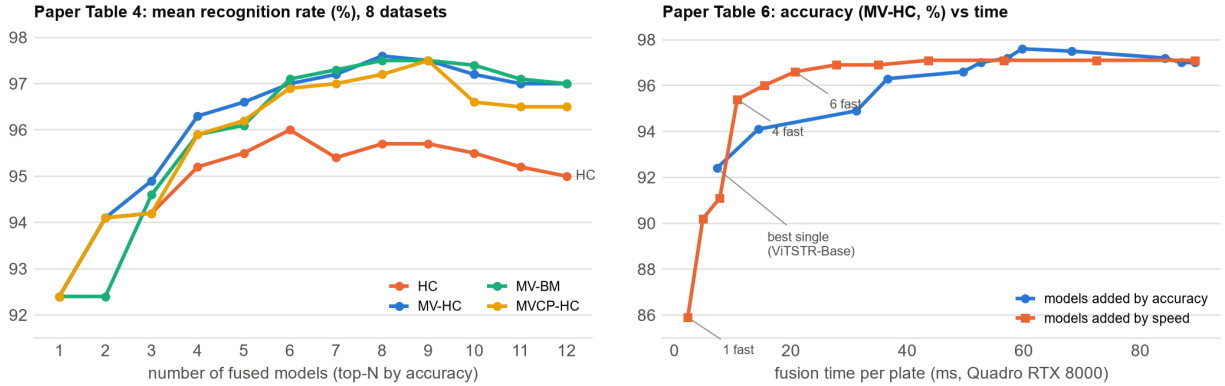


Figure 2. The paper's numbers, redrawn. Left: Table 4 (p.9), fusing the top-N models. Right: Table 6 (p.12), accuracy vs time, models added by accuracy or by speed.

Result	Value	Page
Best single model, 8 datasets (mean)	ViTSTR-Base 92.4 %; STAR-Net 92.2; TRBA 91.1; CR-NET 90.8; ... CRNN 87.1; Multi-Task-LR 85.9	p.9, Table 3
Best fusion, 8 datasets	**MV-HC (top 8) 97.6 %**; MV-BM (top 8) 97.5; MVCP-HC (top 9) 97.5; MVCP-BM 97.2; HC (top 6) 96.0	p.9, Table 3
Worst cases removed	every single model is below 90 % on at least two datasets; every fusion rule is above 90 % on all eight	p.9
Top-N curve (MV-HC)	top 2: 94.1, top 3: 94.9, top 4: 96.3, top 6: 97.0, top 8: 97.6, top 12: 97.0 %	p.9, Table 4
Cross-dataset, best single	Fast-OCR 87.6 %; CR-NET 87.1; STAR-Net 84.4; ViTSTR-Base only 75.9	p.11, Table 5
Cross-dataset, best fusion	**MV-BM (top 8) 90.3 %**; MV-HC 90.1; MVCP-HC 87.6; HC 84.4 (below the best single model)	p.11, Table 5
Hardest unseen set (CD-HARD)	best single 59.6 → MV fusion 65.4 %	p.11, Table 5
Speed, single models	Multi-Task-LR 2.3 ms; Holistic-CNN 2.5; CRNN 2.9; Fast-OCR 3.0; ViTSTR-Base 7.3; TRBA 16.9 ms per plate	p.12, Table 6
Speed, fusion	top 8 by accuracy: 59.7 ms (17 FPS). The 4 / 5 / 6 fastest: 95.4 / 96.0 / 96.6 % at 10.8 / 15.4 / 20.7 ms	p.12, Table 6
Surprise	the 3 fastest fused (91.1 %) are below the best single model (92.4 %)	p.12
Training time, seeds, significance tests	not reported	-

The authors' own explanation (p.10): fusion "does not necessarily lead to significantly improved performance compared to the best model in the ensemble. Instead, it reduces the likelihood of obtaining poor performance", because diverse models make different errors but agree when they are right. HC is weak because every model is also confidently wrong at times (p.11, Fig. 4). Keep this sentence in mind: our run shows exactly when that condition fails.

6. Critical review

Strengths

- **Wide and honest test bed:** 12 models in their original frameworks, 12 public datasets, intra- and cross-dataset results, and detector crops instead of perfect crops (p.5-8).
- **Simple, reusable rules:** no training, about 40 lines of code; anyone can add it to an existing system.
- **Speed is reported next to accuracy** (Table 6), which gives a practical recipe: 4-6 fast models.
- **Useful negative results:** HC is the worst rule and confidence normalisation did not help (p.8, p.11).

Weaknesses (what a reviewer would ask)

- **Test-informed choices.** The "best model" for -BM and the top-N order come from test-set accuracy across datasets. The authors say validation gives nearly the same order (p.9), but strictly every choice should be made on validation only. *We did that in our run.*
- **One training run per model, no confidence intervals, no significance test.** Several gains in Table 4 are differences of 0.1-0.3 points on small test sets (Caltech Cars has 46 plates, UCSD-Stills 60). Our own two runs show why this matters: the same model moved by 1-3 points between runs (section 11).
- **The headline gain partly measures "not knowing which model to use".** We recomputed it from Table 3: if you could pick the best single model *for each dataset*, the mean would already be **96.8 %**. Fusion (97.6 %) adds only **0.8 points** over that. The big +5.2 is against one fixed model (ViTSTR-Base), which is weak on the older US and EU sets. Cross-dataset the per-dataset best is Fast-OCR everywhere (87.6 %), so the +2.7 there is a real gain, mostly on CD-HARD and CLPD.
- **Not every model saw the same data:** CR-NET and Fast-OCR were not trained on RodoSol-ALPR, because they need character boxes (dagger in Table 3, p.9); their 59.0 / 56.7 % there is a data gap, not a model weakness.
- **Heavy for real time:** 8 models cost 59.7 ms per plate on a 48 GB workstation GPU; much slower on a laptop CPU.
- **Latin and Chinese plates only**; no complex script such as Bangla, where one character can be several Unicode code points.
- **Diversity is assumed, not measured.** The paper never reports how often the models fail together, which is exactly what decides whether fusion can work (our finding 2).

Verdict. A careful, peer-reviewed and easy-to-copy study. Its main lesson is about *risk reduction* across datasets rather than a large gain on one dataset, and its claim depends on having many strong and different models. Both points matter for us.

7. What would break on our Bangla plates (and what we did)

Problem	Why	What we did / do
MVCP on Unicode	Bangla vowel signs and conjuncts change the string length, so character positions shift (worse than in the paper)	vote on **tokens** (district, মেট্রো, letter, 6 digits) instead of Unicode characters
Correlated models	all our models learn from the same 5,607 crops; 2,136 of their labels came from a character YOLO (auto-labels), so they can share its mistakes	measure it: pairwise overlap and the oracle (finding 2)
Few strong models	we have one strong reader (char-YOLO); training several strong Bangla OCR models is expensive	test fusion with 2-4 models; teammates' readers can join later (section 15)
Confidence scales differ	a box score (char-YOLO) and a softmax mean (CRNN) do not mean the same thing	check calibration (finding 3); prefer MV-BM over HC
Real time (≥ 10 FPS, RQ3)	running 4 models on every plate adds up	call a second model only when needed (finding 4)
Two-row plates	most Bangla plates have two rows	char-YOLO reads boxes by token kind; CRNN / ViTSTR see the two rows side by side

PART B · OUR EXPERIMENT

8. What we ran and why

No fusion code and most of the paper's datasets are not public (several need a request), so a run on the paper's own data was not possible. Instead the kit asks the paper's question on **our** data: does fusing different OCR models help on Bangladeshi plates, and at what cost? This is the second run. The first one (20 Oct, dataset v1.0) found 19 test "crops" that were whole car photos; the dataset was fixed (v1.1) and everything was trained again from scratch.

Run	Where	Model and settings	Train time
char-YOLOv8n	GPU 0	YOLOv8n from COCO weights; 100 epochs, 320 px, batch 32, SGD, no mirror flips; 101 token classes (the dataset's vocabulary)	52 min
slot CNN	GPU 0 (after)	6-block CNN, 9 slots, 64×128 grey input; 100 epochs, AdamW + OneCycle; 1.73 M parameters	24 min
CRNN-CTC	GPU 1	VGG-style CNN + 2-layer BiLSTM + CTC, 48×256, two rows side by side; 60 epochs; 7.69 M parameters	24 min
ViTSTR-tiny	GPU 1 (after)	ImageNet ViT-tiny (timm), 32×256, first 12 tokens classified; 60 epochs, lr 3e-4; 5.41 M	24 min
fusion	CPU	every subset of 2-4 models × HC, MV-HC, MV-BM, MVCP-HC (paper) and MV-BM+format, fallback (ours); best chosen on validation; exact McNemar test for every row	0.3 min

Item	Setting
Data	498R BanglaLPR All-in-One **v1.1** (downloaded by the notebook from the team Drive), OCR split: 5,607 train crops (3,471 human + 2,136 auto-labelled), 766 validation, 651 test (human labels only). Test sources: LPDBA 202, ANPR 184, KUET 183, NightLPBD 52, BLPD 30.
What v1.1 changed	91 BLPD rows (train 63, val 9, test 19) were whole vehicle photos; they are now cut to the plate. Splits, groups and counts did not change (docs/DATASET_v1_1_CHANGES.md).
Split	grouped 70/15/15, seed 42, no near-duplicate across splits (team rule)
Metric	plate exact (whole plate right after Unicode normalisation, digits as Bangla, no spaces or hyphens), plus CER and character accuracy
Fair choice	MV-BM's "best model", the fallback's first model and every "best" row are chosen on **validation**; test is read once; a tie on validation goes to the faster row. (The paper chose on test.)
Speed	ms per plate, batch 1, on one T4; a fusion costs the sum of its members (run one after another); the fallback costs char-YOLO plus the helpers on the share of plates that need them
Hardware, software	Kaggle Tesla T4 x2, one job per GPU; Python 3.13.15, torch 2.11.0+cu128, Ultralytics 8.4.170, timm 1.0.29

After the run we went further than the notebook (all in `report_src/analysis.py`, no retraining): error overlap between models, bootstrap intervals, confidence calibration, two more fusion rules (weighted votes), a confidence-gated fallback, results per condition and per source, the 19 re-cut plates, and a comparison with the v1.0 run. Every extra rule was again chosen on validation and reported on test.

9. Is the run OK? (health check)

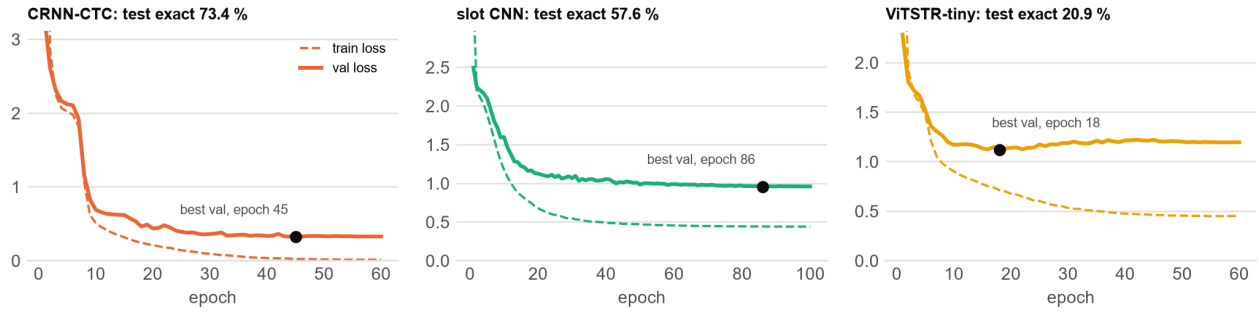


Figure 3. Training (dashed) and validation (solid) loss of the three sequence models (v1.1 run). The black dot is the epoch whose weights were kept.

Check	Result	Meaning
Dataset version	environment.json: dataset_version 1.1; the log shows the download from the "All In One 1.1" Drive folder	the run used the fixed data
Jobs and time caps	all 5 jobs code=0; 79 min wall time on 2 GPUs	nothing failed or was cut short
Code	the 9 kit files in the zip are identical to the current kit (line endings only)	the run is the kit you have
Kaggle numbers = ours	all 66 fusion rows recomputed from the per-plate predictions: identical (val and test); oracle 90.02 = 90.02	the analysis reads the same predictions
Shared models re-scored on this PC	char-YOLO: same text on all 651 test and 766 val plates; CRNN (FP16 copy): 73.43 % test, same as Kaggle; fallback 88.63 %	the GitHub files reproduce the report
No test peeking	best model, tie-break model and every rule chosen on validation	test numbers are fair
char-YOLOv8n	100 / 100 epochs; box mAP50 0.939 on test (P 0.90, R 0.90); character accuracy 97.5 %	healthy (v1.0: mAP50 0.913)
CRNN-CTC	val loss flat from epoch ~45, best at 45; test 73.43 %	healthy but weaker than char-YOLO
ViTSTR-tiny	val loss lowest at epoch 18, then rises; test 20.89 %	**overfits and undertrained:** 1,566 steps at its best epoch vs 300 k iterations in the paper; tiny model, 32-px input
slot CNN	best at epoch 86, val loss 0.96 (v1.0: 1.10); test 57.60 %	**underfits:** 1.7 M parameters from scratch, global max-pool loses position; very sensitive to the run (section 11)
Validation vs test	every model is better on test than on validation (char-YOLO 82.4 vs 86.79)	the validation plates are harder for every model; the ranking is the same on both, so the choices hold
Auto-labels in training	2,136 of 5,607 OCR training labels are auto-labels from a character YOLO	a likely reason why the models share errors (finding 2)
Single run	one seed per model, as in the paper	run-to-run spread is 1-3 points (section 11): add 3 seeds before final claims

> **Verdict: the run is OK and can be quoted.** It is complete, it used v1.1, it matches the kit, and every number in this report is reproducible from the saved predictions and from the shared weights. Its limits are the ones of the design (two weak models, one seed), not errors in the run.

10. Results next to the paper

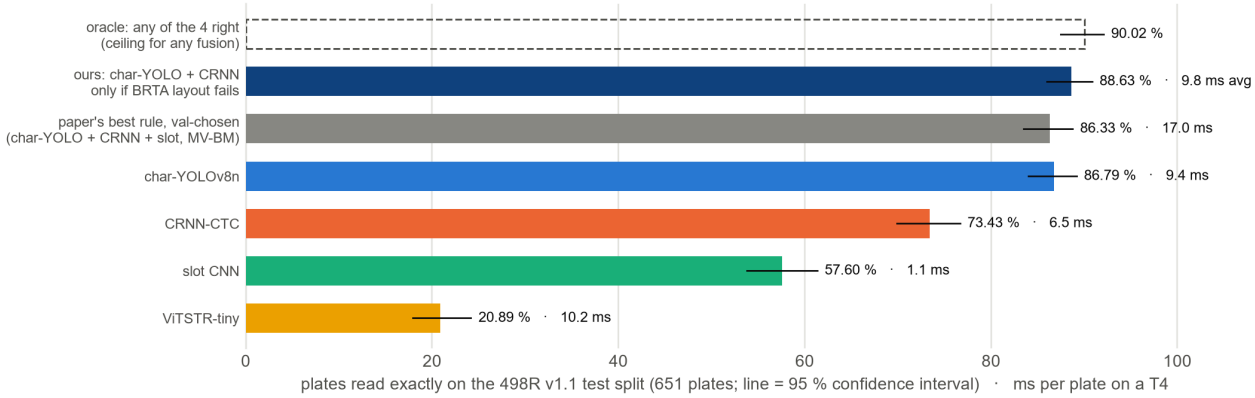


Figure 4. Our test results (651 plates, v1.1). The black lines are 95 % confidence intervals (Wilson): about ±2.6 points for the best model, so a difference of 0.5 points between two readers cannot be read from the bars; the paired McNemar test in section 12 can.

Metric	Paper (page)	Ours (498R v1.1 test)
Best single model, plate exact	92.4 % ViTSTR-Base, mean of 8 sets (p.9)	**86.79 %** char-YOLOv8n (val 82.38 %; 95 % CI 84.0-89.2)
Other single models	85.9-92.2 % (all 12 within 6.5 points)	CRNN 73.43, slot CNN 57.60, ViTSTR-tiny 20.89 % (spread 66 points)
Best fusion with the paper's rules (val-chosen)	97.6 % MV-HC top 8 (+5.2); cross 90.3 % (+2.7)	86.33 %, char-YOLO + CRNN + slot CNN, MV-BM (-0.46; p = 0.58)
HC (most confident)	96.0 % intra; 84.4 % cross, below the best single model	76.2-76.7 % for every subset with char-YOLO and CRNN, 10 points below the best single model
Oracle (any model right)	not reported	90.02 % (ceiling: only +3.23 points)
Fusion time per plate	59.7 ms top 8 (RTX 8000); 10.8 ms for the 4 fastest	17.0 ms for the paper-rule fusion; 27.2 ms for all 4 (T4)
Our format-aware fallback	-	**88.63 %** (+1.84; 12 fixed, 0 broken; p = 0.0005) at 9.8 ms
Training time	not reported	124 GPU-min in total (79 min wall time on 2 GPUs)

Read the gap as a direction, not a contest. The paper used 12 other datasets with Latin and Chinese plates, larger training sets and 12 tuned models. Our numbers are on Bangla plates, with 4 models of which two are undertrained. What carries over is the *pattern*: which rule wins, and when fusion can help at all.

Top-N by validation accuracy (paper Table 4 style)	HC	MV-HC	MV-BM	+ format	fallback	ms (sum)
top 1 (+ char-YOLOv8n)	86.79	86.79	86.79	86.79	**86.79**	9.4
top 2 (+ CRNN-CTC)	76.34	76.34	86.79	88.63	**88.63**	15.8
top 3 (+ slot CNN)	76.65	79.72	86.33	87.40	**88.63**	17.0
top 4 (+ ViTSTR-tiny)	76.50	79.57	86.18	87.10	**88.48**	27.2

The table shows the paper's own pattern on our data: HC and MV-HC fall by about 10 points as soon as CRNN joins; MV-BM stays at or just below the best model; the BRTA filter (+ format) and the fallback lift every row. With two models MV-BM equals the best model alone, as explained in section 4. "ms" is the time for all models of the row; the fallback needs only 9.8 ms on average because CRNN runs on 7.1 % of the plates.

11. What changed from the v1.0 run, and what stayed

The two runs used the same code, the same split and the same settings. Only 182 BLPD rows of the data changed (91 character-box rows and the same 91 OCR rows), and the models were trained again from scratch. The v1.0 per-plate predictions were overwritten, so the comparison below uses the numbers saved in the first analysis (`report_src/v1.0_run/analysis.json`); a paired test between the two runs is not possible.

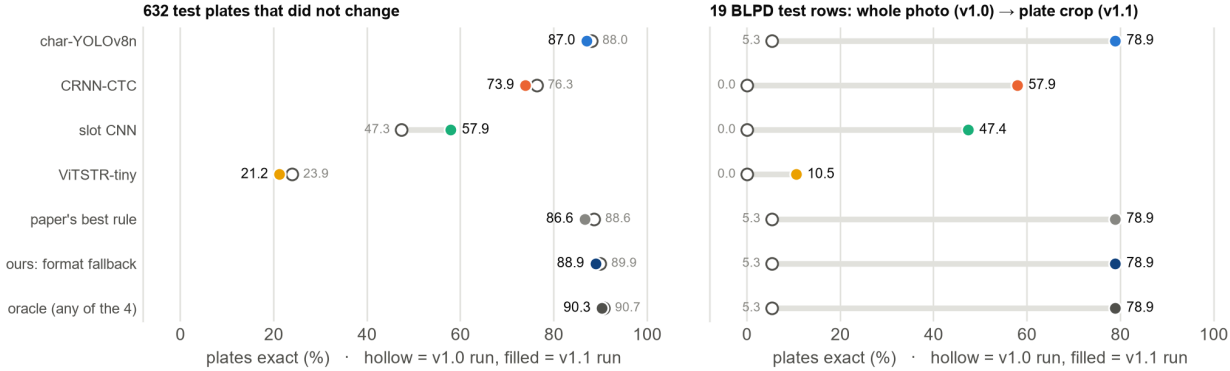


Figure 5. The same readers in both runs. Left: the 632 test plates whose image did not change. Right: the 19 BLPD test rows that were whole car photos in v1.0 and are plate crops in v1.1.



Figure 6. Six of the 19 re-cut BLPD test rows: the v1.0 image (top) and the v1.1 plate crop (bottom), with char-YOLO's v1.1 result.

Plate exact (%)	v1.0 all	v1.1 all	v1.0 632 unchanged	v1.1 632 unchanged	v1.0 19 re-cut	v1.1 19 re-cut
char-YOLOv8n	85.56	**86.79**	88.0	87.0	5.3	78.9
CRNN-CTC	74.04	**73.43**	76.3	73.9	0.0	57.9
slot CNN	45.93	**57.60**	47.3	57.9	0.0	47.4
ViTSTR-tiny	23.20	**20.89**	23.9	21.2	0.0	10.5
paper's best rule	86.18	**86.33**	88.6	86.6	5.3	78.9
format fallback (char-YOLO + CRNN)	87.40	**88.63**	89.9	88.9	5.3	78.9
oracle (any of the 4 right)	88.17	**90.02**	90.7	90.3	5.3	78.9

What the dataset fix did

- On the 19 re-cut rows every reader improved: char-YOLO from 1 plate to 15 (5.3 → 78.9 %). BLPD as a source went from 36.7 to 83.3 %.
- That alone explains the higher overall scores: char-YOLO +1.23 points overall comes from +14 plates on the re-cut rows and -6 on the rest.
- The fusion results on the re-cut rows are all equal to char-YOLO alone: there, no other model rescued a plate.

What two trainings of the same model tell us

On the 632 plates that did not change, the retrained models moved by -1.0 (char-YOLO), -2.4 (CRNN), +10.6 (slot CNN) and -2.7 points (ViTSTR). Only 63 of 5,607 training rows changed, and the slot CNN improved on every source (KUET 60 → 69, LPDBA 55 → 68, ANPR 36 → 47 %), not just where the data changed. So these

are mostly **run-to-run differences** (GPU non-determinism and a different path through training), not effects of the fix. Lesson: **one run cannot rank two readers that are 1-3 points apart**; the slot CNN, trained from scratch, can swing by about 10 points.

What replicated (the findings we can now state with more confidence)

Finding	v1.0 run (20 Oct)	v1.1 run (22 Oct)
Paper's best rule (val-chosen) vs best model	86.18 %, +0.62: 7 fixed / 3 broken, p = 0.34	86.33 %, -0.46: 5 fixed / 8 broken, p = 0.58
Format fallback vs best model	12 fixed / 0 broken, +1.84, p = 0.0005	**12 fixed / 0 broken, +1.84, p = 0.0005**
Layout check as an error detector (test)	64 flagged, precision 100 %	46 flagged, precision 100 %
char-YOLO errors that every model gets wrong	77 of 94	65 of 86
CRNN wrong readings with confidence ≥ 0.9	60 %	65 %
Where the fallback helps most	night (+3.1), NightLPBD (+3.8)	blurry (+4.5); night +0.0 (*did not replicate*)
Main error type of char-YOLO	missed characters (mostly the 19 whole photos)	extra digit boxes and doubled letters (finding 5)

12. Six findings

Finding 1: the paper's recipe is not significant on our data (second time)

The best paper-rule fusion chosen on validation is char-YOLO + CRNN + slot CNN with MV-BM: 82.64 % on validation and 86.33 % on test, against 86.79 % for char-YOLOv8n alone. Per plate, it fixes 5 readings and breaks 8. The exact McNemar test gives p = 0.58, and a bootstrap 95 % interval for the gain is -1.54 to +0.61 points: no evidence of any gain. The choice itself was a tie on validation: char-YOLO + CRNN + ViTSTR with MV-BM also scored 82.64 % and reads 86.79 % on test (exactly char-YOLO); the kit took the faster one. Either way: no gain, at 1.8x the time.

Finding 2: our models fail together, so voting has little to work with

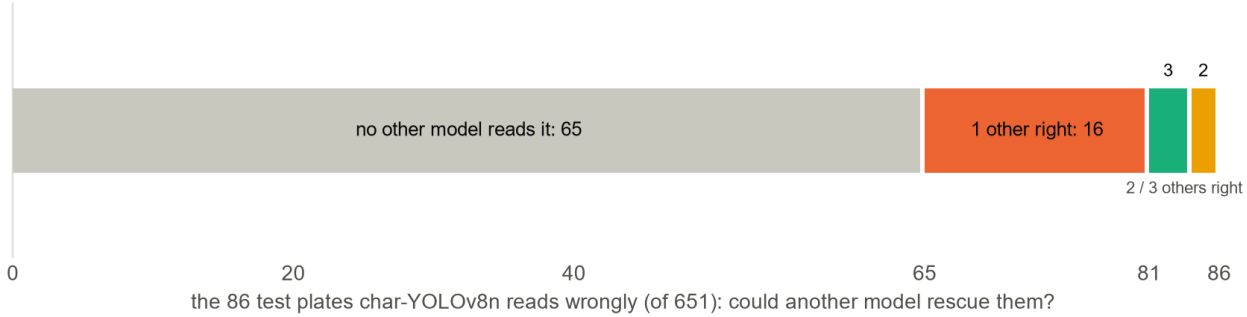


Figure 7. Of the 86 plates char-YOLOv8n reads wrongly, 65 are read wrongly by all four models. Only 21 could be rescued by any rule, so the oracle is 90.02 %.

- When char-YOLO is wrong, CRNN is also wrong 80 % of the time (Yule's Q = 0.90; 0 would mean independent errors, 1 fully shared).
- If three readers each as good as char-YOLO made *independent* errors, a majority vote would be right 95.2 % of the time. Real errors are shared, which is why the vote gives 86.3 %.
- For a wrong plate to be fixed by voting, at least two other models must agree on the right string. That is possible on only 5 plates.
- The vote also *breaks* plates: all 8 it breaks are a series letter that CRNN and the slot CNN both misread the same way (খ→গ / ক, ক→গ, ন→ব, জ→ড), outvoting char-YOLO's right answer (example table after finding 4).
- The paper's 12 models were all strong (85.9-92.4 %), so a wrong model was usually outvoted. Here ViTSTR-tiny and the slot CNN are wrong on most plates and cannot outvote anything.

Finding 3: same confidence, different meaning (why HC fails)

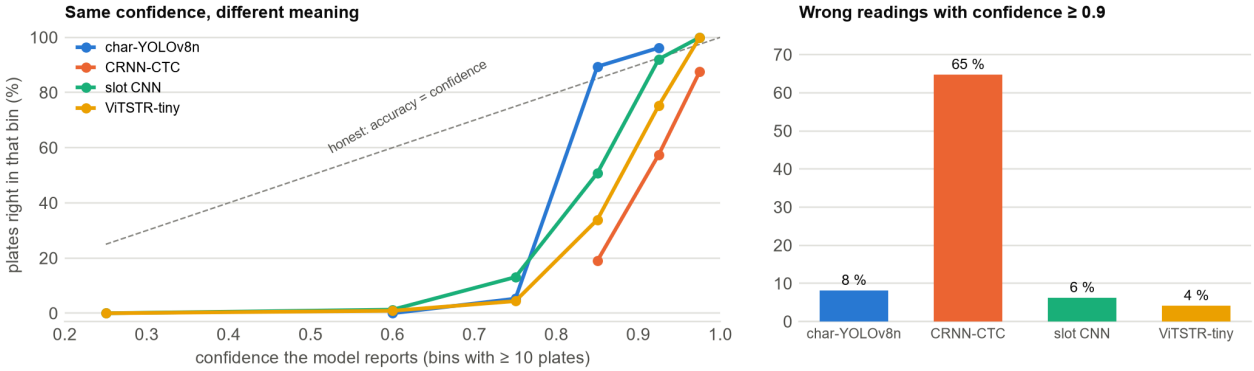


Figure 8. Left: share of right readings in each confidence bin. Char-YOLO's scores are honest above 0.8; CRNN's are not (only 58 % right between 0.9 and 0.95). Right: share of wrong readings with confidence ≥ 0.9.

65 % of CRNN's wrong readings have confidence ≥ 0.9 (mean 0.91 when wrong, 0.97 when right), against 8 % for char-YOLO. So when the two disagree, HC usually believes CRNN and loses: in the char-YOLO + CRNN pair, HC overrules 85 right readings and fixes only 17, and reads 76.3 %, 10 points *below* char-YOLO alone. This is the paper's own finding (p.11), only stronger: within one model the confidence ranks readings reasonably (AUROC 0.84-0.94), but across models the scales do not match.

Finding 4 (our contribution): the BRTA layout is a free and safe fusion signal

Every Bangladeshi plate has one layout: a district, an optional মেট্রো, exactly one letter, then six digits. A reading that breaks it cannot be right. We used this in two ways, both chosen on validation:

- **As an error detector.** 46 of char-YOLO's 651 test readings (7.1 %) break the layout, and **all 46 are wrong** (on validation: all 92 of 92). That is 54 % of its test errors, found without any second model. The readings that pass are 93.4 % right. In fact, on test not one reading of any of the four models that breaks the layout is right.
- **As a fallback rule.** Keep char-YOLO's reading when the layout is valid; otherwise ask CRNN and take its reading if that one is valid. CRNN runs on only 7.1 % of plates, so the mean cost is 9.8 ms instead of 15.8 ms. Result: **88.63 %**, 12 plates fixed and **none broken** (McNemar p = 0.0005; bootstrap 95 % interval of the gain +0.92 to +2.92 points). Because it only replaces readings that are certainly wrong, it cannot break a right one.
- **What happens to the 46 flagged plates:** CRNN fixes 12; on 10 CRNN's reading is invalid too, so char-YOLO's (wrong) reading stays; on 24 CRNN gives a valid but wrong reading. Those plates were wrong before and stay wrong, so nothing is lost, but they show where a better second reader would pay off.
- **Kaggle's headline row adds ViTSTR as a third helper** (val 85.12 vs 84.99 %: one validation plate); on test both read 88.63 % and never differ on a right answer. We ship the two-model version: one model less and 0.7 ms faster, with no test number changed.
- **Confidence gates add nothing:** also asking CRNN when char-YOLO's confidence is below a threshold (0.5-0.95) did not raise the validation score, so the rule uses the layout alone. The same filter inside the vote (MV-BM+format) also reaches 88.63 % but always runs both models (15.8 ms).

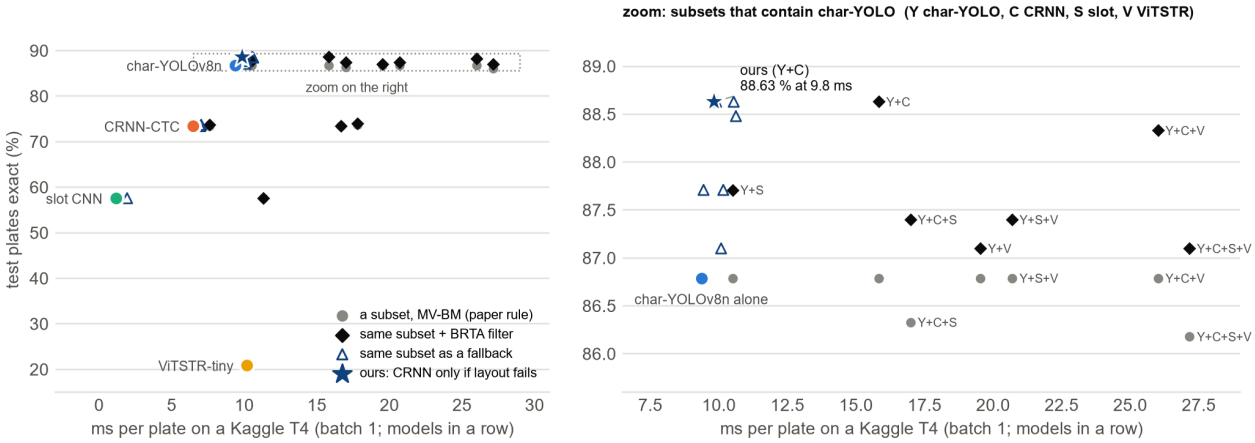
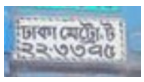


Figure 9. Accuracy vs time for every subset (paper Table 6 style). Grey: the paper's MV-BM rule; black diamonds: the same subset with the BRTA filter; open triangles: the same subset as a fallback; star: our fallback (char-YOLO, then CRNN only when the layout fails), the cheapest point on the top edge.

Examples (test crops; red = wrong; small number = confidence)

Crop	Truth	char-YOLOv8n	CRNN	Our fallback
	ঢাকা মেট্রো-ঘ ১৭-৩১০৯	ঢাকা মেট্রো-গঘ ১৭-৩১০৯ 0.82	ঢাকা মেট্রো-ঘ ১৭-৩১০৯ 1.00	ঢাকা মেট্রো-ঘ ১৭-৩১০৯
	ঢাকা মেট্রো-ট ২২-৩৩৭৫	ঢাকা মেট্রো-হট ২৩৩৭৫ 0.85	ঢাকা মেট্রো-ট ২২-৩৩৭৫ 0.98	ঢাকা মেট্রো-ট ২২-৩৩৭৫
	ঢাকা মেট্রো-ব ১১-৫৫৩৫	ঢাকা মেট্রো-ব ১১৫৩৫ 0.92	ঢাকা মেট্রো-ব ১১-৫৫৩৫ 0.97	ঢাকা মেট্রো-ব ১১-৫৫৩৫
	খুলনা মেট্রো-ট ১১-০৮৯৯	খুলনা মেট্রো-ট ১১০৮৯১৯৯২ 0.72	খুলনা মেট্রো-ট ১১-০৮৯৯ 0.95	খুলনা মেট্রো-ট ১১-০৮৯৯

Above: plates the fallback fixes. Char-YOLO finds a letter twice under two classes (গঘ, হট), loses a digit, or boxes digits twice (up to nine digits); each breaks the layout, so CRNN is asked and its valid reading is kept. Below: plates the paper's vote *breaks*. CRNN and the slot CNN make the same letter mistake, so two votes beat char-YOLO's right answer; our fallback never asks them because char-YOLO's reading is valid.

Crop	Truth	char-YOLOv8n	CRNN	slot CNN	paper vote	our fallback
	চট্ট মেট্রো-খ ১১-০৭৩০	চট্ট মেট্রো-খ ১১-০৭৩০ 0.90	চট্ট মেট্রো-ক ১১-০৭৩০ 0.94	চট্ট মেট্রো-ক ১১-০৭৩০ 0.81	চট্ট মেট্রো-ক ১১-০৭৩০	চট্ট মেট্রো-খ ১১-০৭৩০
	খুলনা মেট্রো-ন ১১-০৬৩৫	খুলনা মেট্রো-ন ১১-০৬৩৫ 0.89	খুলনা মেট্রো-ব ১১-০৬৩৫ 0.95	খুলনা মেট্রো-ব ১১-০৬৩৫ 0.77	খুলনা মেট্রো-ব ১১-০৬৩৫	খুলনা মেট্রো-ন ১১-০৬৩৫
	ঢাকা মেট্রো-খ ১২-২৯২৯	ঢাকা মেট্রো-খ ১২-২৯২৯ 0.90	ঢাকা মেট্রো-গ ১২-২৯২৯ 0.96	ঢাকা মেট্রো-গ ১২-২৯২৯ 0.86	ঢাকা মেট্রো-গ ১২-২৯২৯	ঢাকা মেট্রো-খ ১২-২৯২৯
	ঢাকা মেট্রো-খ ১৩-৮৫২৮	ঢাকা মেট্রো-খ ১৩-৮৫২৮ 0.85	ঢাকা মেট্রো-গ ১৩-৮৫২৮ 0.99	ঢাকা মেট্রো-গ ১৩-৮৫২৮ 0.87	ঢাকা মেট্রো-গ ১৩-৮৫২৮	ঢাকা মেট্রো-খ ১৩-৮৫২৮

Finding 5: on real crops, char-YOLO's errors are doubled boxes, so the next fix is its decoder

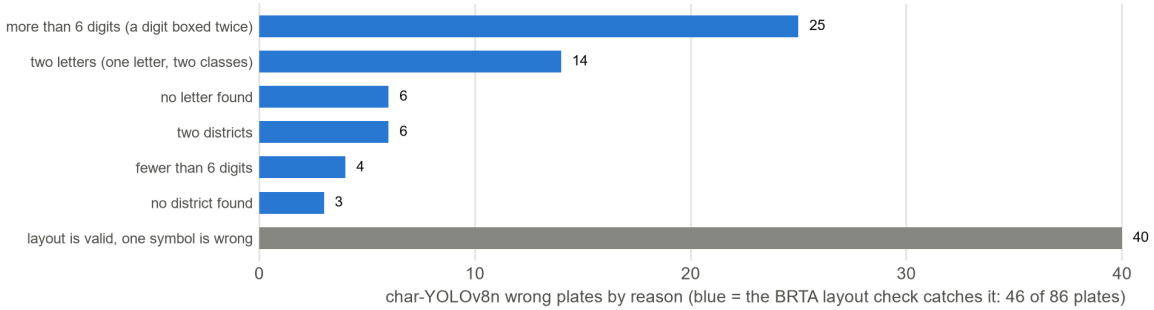


Figure 10. Why char-YOLOv8n's 86 wrong test readings are wrong (a plate can have several reasons). Blue: the layout check catches it.

In v1.0 most of char-YOLO's errors looked like *missed* characters; most of those came from the 19 whole photos. On real crops the picture is different: of the 147 token errors, 79 are wrong symbols, 52 are **extra** tokens and only 16 are missing ones. The layout check catches the extra and doubled ones (46 plates). The other 40 wrong plates have a valid layout with one wrong symbol (mostly a letter or a digit confused with a similar one); no rule can catch those without a second opinion. A decoder that keeps **one letter** (the most confident) and **six digits** (merge boxes that overlap) would target 39 of the 86 wrong plates before any fusion.

Finding 6: where it helps (conditions and sources)

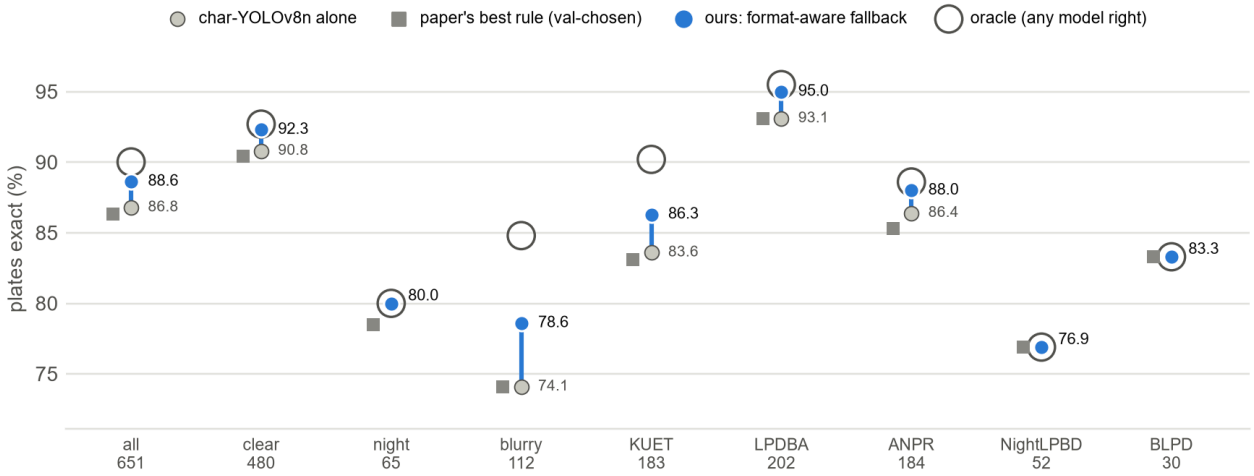


Figure 11. Plate exact by condition and source. Grey dot: char-YOLO alone; square: the paper's val-chosen vote; blue dot: our fallback; ring: oracle. Where the blue dot sits in the ring, our rule already reaches what any fusion of these four models could reach.

Group (plates)	char-YOLO	CRNN	slot	ViTSTR	paper vote	our fallback	oracle
clear (480)	90.8	77.5	62.3	23.3	90.4	**92.3**	92.7
night (65)	80.0	61.5	27.7	4.6	78.5	**80.0**	80.0
blurry (112)	74.1	64.3	54.5	19.6	74.1	**78.6**	84.8
KUET (183)	83.6	75.4	68.9	42.1	83.1	**86.3**	90.2
LPDBA (202)	93.1	81.7	68.3	20.8	93.1	**95.0**	95.5
ANPR (184)	86.4	68.5	46.7	7.1	85.3	**88.0**	88.6
NightLPBD (52)	76.9	59.6	25.0	3.8	76.9	**76.9**	76.9
BLPD (30)	83.3	60.0	40.0	6.7	83.3	**83.3**	83.3

- The fallback helps most on **blurry** plates (+4.5 points) and on KUET (+2.7); there the oracle is still 6.2 / 3.9 points higher, so a stronger second reader would help most there.
- **Night: no gain this time** (80.0 % for char-YOLO, the fallback and the oracle). In v1.0 night was the best case (+3.1). With 65 night plates one plate is 1.5 points, so per-group gains of 2-3 points are not stable between runs; quote the overall test, and per group only the direction.
- The paper's vote is at or below char-YOLO alone in every group.
- Weather tests from the notebook (char-YOLO alone): real fog copies 80.8 % (26 plates), synthetic rain 75.9 %. CRNN drops to 50 % in fog, so it cannot help there.

13. Speed: does it fit our real-time target?

Where	char-YOLO alone	Our fallback (avg per plate)	Plates per second
Kaggle T4 (kit timing, batch 1, model step)	9.36 ms	9.82 ms (CRNN on 7.1 %)	≈ 102
This PC, NVIDIA GeForce RTX 5060 Ti (whole reader: preprocess, model, decode, layout check)	5.49 ms	5.82 ms (median 5.22)	≈ 172
This PC, CPU only (Intel(R) Core(TM) Ultra 7 265K, 8 threads)	11.39 ms	12.06 ms (median 10.99)	≈ 83

Local timing: the first 200 test crops, already in memory, after 20 warm-up plates; CRNN ran on 12 of them. This is the OCR step only (plate detection is not included). The CPU is a fast desktop processor; a laptop CPU will be slower, so measure the full pipeline on the target machine before claiming RQ3.

14. Ours vs the proposal's targets

Proposal target	Where we are (this run)	Status
Character recognition accuracy ≥ 90 % on test crops (RQ2)	char-YOLOv8n 97.5 %; with the fallback 97.3 %	met at character level
Plate read exactly (our stricter view)	86.79 % → 88.63 % with the fallback (95 % CI of the plate rate about ±2.4)	close to 90 %; next fixes in section 20
≥ 10 FPS full pipeline (RQ3, O4)	OCR step 9.8 ms per plate on a T4, 12.1 ms on a desktop CPU	OCR is not the bottleneck; full pipeline not yet timed
BRTA-format-aware correction (O3)	layout check finds 54 % of char-YOLO's errors with 100 % precision; fallback +1.84 points	first working piece, replicated
Multi-frame voting (RQ3, O4)	not tested here (images only)	this paper's rule + our filter, next

15. How to use this paper in our research

Use	What to do	Links to
1. Final OCR choice	Use char-YOLO as the main reader and CRNN as a fallback when the BRTA layout fails (88.63 %, 9.8 ms). Report it as plate exact vs ms per plate, like the paper's Table 6 (our Figure 9).	O3, RQ2
2. Grammar-aware fusion	Make the BRTA layout part of every fusion rule. It is the Bangla-specific part the paper does not have, and it never hurts a right reading.	O3 (format-aware correction)
3. Multi-frame voting in video	The same rule, but the "models" are the frames of one tracked vehicle: drop invalid readings, take the majority string, stop OCR once 3 valid readings agree. The proposal already cites this paper for it.	RQ3, O4
4. A team ensemble for free	p24_fusion.py accepts any run folder with ocr_preds_val.csv and ocr_preds_test.csv on the same split. The PARSeq (#31), VLM (#22) and ved.py scripts write both files, so teammates' readers can join without retraining. Tried on 24 Oct with paper #2's character model as the helper: 88.79 % (13 fixed / 0 broken), and the oracle rises to 91.71 %.	O5, team work
5. A reporting standard	Choose on validation, report test once, give the confidence interval, use a McNemar test for "A beats B", and run more than one seed. This answers the weak points we found in the paper and the run-to-run spread we measured.	all RQs
6. Robustness argument	Show per-condition results (night, blur, fog) for single vs fused readers: the paper's real claim is fewer bad cases, which is what RQ4 asks. Report direction per group, not small per-group gains.	RQ4

16. What benefit do you get from it?

- **A real, significant and replicated gain at almost no cost:** +1.84 points plate exact (12 fixed, 0 broken, p = 0.0005) in two independent trainings, for +0.46 ms per plate and no new training.
- **A free error detector:** the layout check flags 7 % of readings, all of them wrong. In a live system you can reject them or wait for the next video frame; accepted readings are 93.4 % right.
- **A clear answer to "why not just ensemble everything?"** We can show with numbers (oracle, error overlap, calibration) when fusion helps and when it does not. Faculty like a negative result that is explained.
- **A methods contribution for the final report:** grammar-aware fusion for Bangla plates, with fair validation-based selection, significance tests and a replication, which the original paper lacks.
- **Direction for the next experiments:** the oracle says better voters are needed, the error breakdown says the char-YOLO decoder should keep one letter and six digits, the calibration plot says never to use HC, and the run-to-run spread says to use seeds.
- **A dataset fix for the whole team:** every teammate's OCR test contained the BLPD whole photos; v1.1 fixed them and this run confirms it (char-YOLO 5 → 79 % on those rows).
- **Ready-to-use weights** for the team: char-YOLOv8n and the CRNN fallback on GitHub, tested on this PC (section 21).
- **Literature value:** a peer-reviewed reference (CIARP, LNCS) for both model fusion and multi-frame voting in the related-work section.

17. What to show the faculty (9 slides)

#	Slide title	What is on it	Source in this report
1	The question	single OCR models are uneven; can combining several help? how much? which?	section 1
2	The paper's method	12 models, HC / MV / MVCP, tie-breaks	Figure 1 A
3	The paper's results	92.4 → 97.6 % (8 sets), 87.6 → 90.3 % (unseen sets), HC worst, 4-6 fast models	Figure 2, section 5
4	Our critique	chosen on test; one run; per-dataset best already 96.8 %, so fusion adds 0.8 there	section 6
5	Our experiment	4 Bangla OCR models, dataset v1.1, val-chosen; best single 86.79 %, paper's vote 86.33 % (n.s.)	Figure 4

6	Why plain fusion fails here	65 of 86 errors shared by all; CRNN overconfident (65 %)	Figures 7 and 8
7	Our fix: BRTA-aware fallback	88.63 % (+1.84, 12 / 0, p = 0.0005) at 9.8 ms; every invalid reading is wrong	Figure 1 B, Figure 9, example table
8	Replication on the fixed dataset	same gain and same 12 / 0 in both runs; the 19 re-cut BLPD rows 5 → 79 %; run-to-run spread 1-3 points	Figures 5 and 6, section 11
9	What it means for our system	final OCR, decoder fix, video voting, seeds, team ensemble	sections 15 and 20

> **Keep it honest:** say "on our test split" for every number of ours, say that two of our four models are undertrained, that all choices were made on validation, and that each number comes from one training run (two runs for the main result). Do not compare 88.6 % with the paper's 97.6 % as if it were the same test.

18. What to explain: a 3-minute script

"This paper, from the Brazilian group that also wrote our paper #23 on near-duplicates, asks a simple question: if we read the same plate with several different OCR models and combine the answers, do we read better? They trained 12 well-known models on 8 public datasets and combined them with three rules: trust the most confident model, take the majority string, or vote character by character. The majority string wins. The mean accuracy goes from 92.4 to 97.6 percent, and on unseen datasets from 87.6 to 90.3. Trusting the most confident model is the worst rule, because models are often confidently wrong. And four to six fast models give most of the gain.

When we read the tables closely, we found two weak points. They picked the best models on test results, and part of the gain only replaces knowing which model is best for each dataset: per dataset, the best single model already averages 96.8 percent.

We repeated the idea on Bangla plates with four different models trained on our merged dataset, and we chose everything on validation. We ran it twice: once before and once after we fixed 19 test images that were whole car photos. Both times, the paper's voting gave no significant gain; this time it was even 0.5 points lower. The reason is measurable: on 65 of the 86 plates our best model gets wrong, every other model is wrong too. Fusion cannot invent an answer that no model has. We also saw the paper's confidence problem: CRNN gives 65 percent of its wrong answers a confidence above 0.9.

Then we used something the paper does not have: the Bangladeshi plate format. A reading must have a district, one letter and six digits. Every reading that breaks this rule was wrong. So we keep our character-YOLO reading when it is valid, and ask CRNN only for the 7 percent that are invalid. That gives 88.6 percent, plus 1.8 points, with 12 plates fixed and none broken, which is significant, and it costs less than half a millisecond per plate on average. We got exactly the same gain in both runs.

For our project this means three things: the final reader is character-YOLO plus a format-aware fallback; the same format filter and majority vote will combine readings over video frames; and to gain more, we should fix how character-YOLO turns boxes into text and add stronger, more different second readers, with several seeds."

19. Questions the faculty may ask

Q1. The paper gets +5 points from fusion. Why do you get nothing?
Fusion needs several strong models that make different mistakes. The paper had 12 models between 86 and 92 %. We have one strong model, one medium and two weak ones, and their errors overlap: the oracle (any model right) is only 90.0 %, so no voting rule can give more than +3.2. Also, part of the paper's +5 is against one fixed model; against the best model per dataset the gain is 0.8.

Q2. Isn't +1.8 points also noise?
No. On the same 651 plates the fallback fixes 12 and breaks 0; the exact McNemar test gives p = 0.0005 and the bootstrap interval of the gain is +0.9 to +2.9 points. A second training on the corrected data gave the same 12 / 0. The paper's rule (5 fixed, 8 broken) gives p = 0.58.

Q3. Your numbers changed between the two runs. Which do we trust?
The v1.1 run: it uses the corrected data. Overall scores rose because the 19 whole photos became crops. On the 632 unchanged plates single models moved by 1-3 points (slot CNN about 10), which is normal run-to-run variation. That is why we test gains on the same plates (McNemar) and why the next step is 3 seeds. The fallback's gain did not move.

Q4. Did you tune on the test set?
No. The best model, the tie-break model, the rule and the subset were all chosen on the 766 validation plates; the test numbers were read once. This is stricter than the paper, which ranked models on test results.

Q5. Is the format check cheating? It knows what plates look like.
It uses only the public BRTA format, never the label of the plate. It is the same knowledge a human reader has. For a plate outside the format (army, diplomatic, old plates), char-YOLO's reading breaks the layout anyway; the rule then keeps it unless CRNN produces a valid BRTA string. Such plates are not in our test set, so this case is not measured yet.

Q6. Why are ViTSTR and the slot CNN so weak?
They were trained for about 8,700 (slot) and 5,220 (ViTSTR) steps on 5,600 crops, against 300,000 iterations on tens of thousands of images in the paper. ViTSTR starts to overfit after epoch 18. We did not tune them because the question was fusion; the next step is to train them longer with synthetic plates, or replace them with PARSeq from paper #31.

Q7. Why did the gain at night disappear?
There are only 65 night plates, so one plate is 1.5 points. In v1.0 the fallback fixed two night plates. In v1.1 no other model reads any night plate that char-YOLO misses (the oracle at night equals char-YOLO), so there is nothing for a fallback to fix. Gains on small groups move between runs; the overall gain did not.

Q8. Why not calibrate the confidences and use HC?
The paper tried normalising confidences and it did not help. Temperature scaling on validation is a cheap test we can add, but MV-BM plus the format check already avoids the problem.

Q9. How does this help with video?
A vehicle is seen in many frames. The same rule combines the frames: drop invalid readings, take the majority string, and stop reading once three valid readings agree. That saves time and is our RQ3 and O4.

Q10. Do you reach the 90 % target?
Character accuracy is 97.3 %, above the target. Whole-plate accuracy is 88.6 % with the fallback; the decoder fix in section 20 targets 39 of the 86 remaining wrong plates.

20. Next steps (in order)

1. **Decoder fix in char-YOLO (O3):** keep one letter (the most confident), keep six digits (merge boxes that overlap, class-agnostic), retry with a lower box threshold when a district or letter is missing. It targets 25 plates with extra digits and 14 with a doubled letter. Test it offline on the saved best.pt (no retraining), validation first.
2. **Seeds:** 3 seeds for char-YOLO and CRNN, so the final table has mean ± spread (we measured 1-3 points between runs).
3. **Better second readers:** PARSeq (#31) or a longer-trained ViTSTR with synthetic plates; re-run p24_fusion.py with them and watch the oracle (now 90.02 %). Fusion only pays when it rises.
4. **Video:** apply the format filter + majority vote over tracked frames and measure video-level accuracy and FPS (RQ3).
5. **Team table:** done on 24 Oct (add_results.py, 4 v1.1 rows). There our char-YOLOv8n (86.79 %) and paper #2's YOLOv12n character model (87.71 %) are tied on the same plates (24 vs 18 plates, McNemar p = 0.44).

21. Files: GitHub and Drive

Where	File	What
GitHub upload folder	`weights/paper24/MD.Rokib.Hasan.Oli/`	prepared in `After Train/github_upload/`; upload the folder as it is
	`best_chars_v8n.pt`	char-YOLOv8n, the run's best.pt, unchanged (6.9 MB)
	`best_crnn_fp16.pt`	CRNN-CTC weights stored in FP16 + its token list and input size (15.4 MB; the FP32 file is 30.8 MB, over GitHub's 25 MB web limit); same accuracy, tested
	`paper24_results.zip`	the run's results without weights: predictions, results.json, fusion table, logs, code
	`results_vs_paper.md`	the kit's ours-vs-paper table from the run
	`Paper24_Study_Report.pdf`	this report
	`README.md`	what is in the folder, quick start (tested), results, checksums
Team Drive	`498R / weights / paper24 / paper24_outputs.zip`	everything incl. all four models' weights (124 MB); prepared in `After Train/for_drive/`
This PC	`After Train/results.zip`	the full Kaggle output (298 MB); the v1.0 output was overwritten by it
This PC	`report_src/`	rebuild: `analysis.py` → `local_check.py` → `charts.py` → `build_report.py`; v1.0 sources in `v1.0_run/`

Appendix: where every number comes from

Number	Source
Paper numbers	arXiv 2309.04331v2: Table 3 (p.9), Table 4 (p.9), Table 5 (p.11), Table 6 (p.12), text p.2-12 (re-read 24 Oct 2026)
Per-dataset-best 96.8 % / 87.6 %	our arithmetic on Table 3 and Table 5 (best value per column, then the mean)
Single models, fusion table, oracle	Kaggle run 22 Oct 2026 (v1.1): results.zip → runs/*/results.json, runs/fusion/results.json
Overlap, McNemar, bootstrap, calibration, extra rules, groups	analysis.py on runs/*/ocr_preds_{val,test}.csv → analysis.json
v1.0 numbers	the first analysis (20-21 Oct): report_src/v1.0_run/analysis.json
Conditions, training-label counts, re-cut rows	ocr/test_conditions.csv and ocr/train.csv in BanglaLPR_AllInOne_v1_1.zip; dataset/docs/blpd_crop_fix.md
Local re-score and timing	local_check.py on this PC (NVIDIA GeForce RTX 5060 Ti, Intel(R) Core(TM) Ultra 7 265K), 24 Oct 2026 → local_check.json